ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 2

BASIC PRINCIPLES OF COMPUTER PROGRAMMING AND COMPUTER LITERACY

PRACTICE EXAM 2 · MARKING GUIDELINE

MARKS: 100

Marking guideline for the Academic Central NC(V) Level 2 Basic Principles of Computer Programming and Computer Literacy practice exam 2.

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 2. It is not a Department of Higher Education and Training marking guideline.

How to mark: each ✓ is one mark. Award marks for correct logic even if there are small syntax slips. Accept any correct equivalent code, variable names and flowchart layout.

SECTION A

QUESTION 1

1.1 1.1.1 B ✓ 1.1.2 C ✓ 1.1.3 A ✓ 1.1.4 B ✓ 1.1.5 D ✓ 1.1.6 A ✓ 1.1.7 C ✓ 1.1.8 C ✓ 1.1.9 A ✓ 1.1.10 D ✓ (10)

1.2 1.2.1 False ✓ 1.2.2 True ✓ 1.2.3 True ✓ 1.2.4 True ✓ 1.2.5 False ✓ 1.2.6 True ✓ 1.2.7 True ✓ 1.2.8 True ✓ 1.2.9 True ✓ 1.2.10 False ✓ (10)

1.3 1.3.1 H ✓ 1.3.2 I ✓ 1.3.3 A ✓ 1.3.4 F ✓ 1.3.5 G ✓ 1.3.6 B ✓ 1.3.7 D ✓ 1.3.8 C ✓ 1.3.9 E ✓ 1.3.10 J ✓ (10)

TOTAL SECTION A: [30]

SECTION B

QUESTION 2

2.1 Input (any TWO) ✓✓: keyboard, mouse, scanner, microphone, webcam. Output (any TWO) ✓✓: monitor, printer, speakers, projector. Storage (any TWO) ✓✓: hard drive, SSD, flash drive, SD card, DVD (6)

2.2 System software runs and manages the computer and hardware ✓, e.g. Windows, Linux, Raspberry Pi OS, drivers ✓; application software lets the user do specific tasks ✓, e.g. Word, Excel, a browser, a game ✓ (4)

2.3 Compiling translates the source code into machine code and checks for syntax errors ✓ – produces an object file (.o) ✓; building compiles and links the code with libraries into a program that can run ✓ – produces an executable file (.exe / binary) ✓ (4)

2.4 Syntax error – breaks the rules of the language (e.g. missing semicolon), program does not compile ✓✓; runtime error – program crashes while running (e.g. dividing by zero) ✓✓; logic error – program runs but gives the wrong result (e.g. + instead of −) ✓✓ (6)

[20]

QUESTION 3

3.1.1 It includes the standard input/output library so that printf() and scanf() can be used ✓. (1)

3.1.2 Voltage, current and resistance can have decimal values ✓. (1)

3.1.3 The address of the variable – scanf stores the value typed into that variable ✓. (1)

3.1.4 12 ÷ 0,5 = 24 ✓ → Resistance = 24.00 ohm ✓ (2)

3.1.5 Division by zero gives a runtime error / meaningless result ✓. Code: condition ✓, error message ✓, calculation only in the else ✓, e.g. (4)

```
if (current == 0) {
    printf("Current cannot be zero\n");
} else {
    resistance = voltage / current;
    printf("Resistance = %.2f ohm\n", resistance);
}
```

3.2 #include and main ✓, variables declared (float) ✓, prompts with printf ✓✓, scanf for both values ✓, correct calculation × 5 ✓, output with %.2f ✓, return 0 / correct structure ✓, e.g. (8)

```
#include <stdio.h>

int main(void) {
    float hours, costPerHour, weekCost;
    printf("Hours worked per day: ");
    scanf("%f", &hours);
    printf("Cost per hour (R): ");
    scanf("%f", &costPerHour);
    weekCost = hours * costPerHour * 5;
    printf("Cost for the week: R%.2f\n", weekCost);
    return 0;
}
```

[17]

QUESTION 4

4.1.1 Correct start values ✓, i from 1 to 5 ✓, total unchanged for odd i ✓, total 2 at i = 2 ✓, total 6 at i = 4 ✓ (5)

i	i % 2 == 0?	total
–	–	0
1	no	0
2	yes	2
3	no	2
4	yes	6
5	no	6

4.1.2 6 ✓ (1)

4.1.3 i = 1 before the loop ✓, while (i <= 5) ✓, same if-statement inside ✓, i++ inside the loop ✓ (4)

```
i = 1;
while (i <= 5) {
    if (i % 2 == 0) {
        total = total + i;
    }
    i++;
}
```

4.2 Declare and read the mark ✓, invalid check first (mark < 0 || mark > 100) ✓✓, ≥ 80 Distinction ✓, ≥ 50 Pass ✓, else Fail ✓, correct use of if / else if ✓ (7)

```
int mark;
printf("Enter the mark: ");
scanf("%d", &mark);
if (mark < 0 || mark > 100) {
    printf("Invalid mark\n");
} else if (mark >= 80) {
    printf("Distinction\n");
} else if (mark >= 50) {
    printf("Pass\n");
} else {
    printf("Fail\n");
}
```

[17]

QUESTION 5

5.1 when green flag clicked ✓; forever ✓; if gpio 4 is high / button pressed (with pull-up: gpio 4 is low) ✓; set gpio 18 to output high ✓; else ✓; set gpio 18 to output low ✓ (6)

5.2 Any TWO ✓✓: no syntax errors – blocks only fit where they are allowed; visual and easy to understand; focus on logic and problem solving; quick results with hardware motivate learners (2)

5.3 Start ✓; red on ✓, wait 5 s ✓, red off / green on ✓, wait 4 s ✓, green off / amber on ✓, wait 2 s and amber off ✓, arrow back to red (repeat forever, no stop) ✓ (8)

[16]

TOTAL SECTION B: [70]

TOTAL: 100

END OF MARKING GUIDELINE